SYSTEM DEVELOPMENT ROADMAP

AND TECHNICAL IMPLEMENTATION PLAN

Psychometric-Based College Course Recommendation System Using Student Profiling

React-Based Web Application

Recommended architecture: React + Laravel REST API + PostgreSQL

Prepared for the Capstone Project Team
Tagoloan Community College

Based on the uploaded Capstone Project Proposal (May 2026)

Prepared: 24 July 2026

Document Purpose and Basis

This document converts the capstone proposal into an actionable system-development plan. It preserves the proposal's core purpose - combining admission examination results with a RIASEC vocational-interest assessment to generate ranked college-course recommendations - while adapting the implementation to a React-based web interface.

The proposal identifies the major modules as user management, student profiling, RIASEC assessment, course recommendation, and report generation. It also describes an Agile flow of planning, design, development, testing, deployment, review, and launch. This roadmap expands those requirements into a buildable product scope, database design, API plan, recommendation logic, test strategy, deployment plan, and 18-week implementation schedule.

> **Important implementation note**
> React is the presentation layer, not the complete system. A secure backend API, database, recommendation engine, authentication, report generator, and deployment environment are still required. The recommended plan keeps Laravel as the backend to stay close to the proposal, while React replaces the original Bootstrap-only front end.

Proposal basis: Chapter 1 (purpose, objectives, scope), Chapter 3 (Agile methodology, architecture, testing and deployment), and Appendix C (42-item RIASEC test).

Contents

1. Executive Analysis and Technical Decisions

1.1 What the system must accomplish

The system is a decision-support platform for incoming Tagoloan Community College applicants. It should systematically collect applicant information, use a verified admission examination result as an indicator of academic competence, measure vocational interest through RIASEC, and rank courses that fit both the student's interest profile and institutional admission rules.

Area	Source-derived requirement	Implementation interpretation
Primary users	Incoming students and Guidance/Admission personnel	Student portal plus role-based administrative portal
Core inputs	Student profile, admission examination score, RIASEC answers	Validated forms, staff-verified exam data, versioned assessment responses
Core processing	RIASEC scoring, admission evaluation, machine learning/course matching	Explainable rules first; optional ML after dataset validation
Core outputs	Dashboard, psychometric profile, top courses, guides and reports	Ranked recommendations with reasons, eligibility status, printable report and audit trail
Institutional scale	Approximately 8,000 applications annually	Design for batch imports, indexed queries, and concurrent online use
Scope boundary	Incoming TCC students; TCC programs only; guidance, not final choice	No external courses, no automatic enrollment, no long-term outcome tracking in MVP

1.2 Critical gaps found in the proposal

Gap	Why it matters	Required resolution
React is not in the written technology stack	The proposal lists HTML/CSS/JavaScript/Bootstrap and Laravel.	Use React for the presentation layer; keep Laravel as REST API. Update Chapters 1 and 3 accordingly.
Database conflict	Development text says MySQL/phpMyAdmin, while the architecture says PostgreSQL.	Select one database. This roadmap recommends PostgreSQL. Use MySQL only if hosting or team capability requires it, and revise every diagram and chapter consistently.
Machine-learning dataset is undefined	Decision Tree and Random Forest are named, but labels, features, sample size and metrics are not specified.	Deliver an explainable rules baseline first. Train ML only on psychometrician-approved labeled data; disclose limitations.
Admission score ownership is unclear	Architecture implies students can input admission scores.	Students may view but should not approve official scores. Guidance/Testing staff must encode, import or verify them.
Mobile app appears in architecture	The scope describes a web-based system, but the figure refers to web/mobile.	Treat the project as responsive web only. A native mobile app is future scope.
Recommendation policy is underspecified	The 2.50 threshold is mentioned, but course-specific rules and direction of grading must be confirmed.	Create configurable course admission rules and obtain signed validation from TCC personnel.
RIASEC instrument governance is missing	Appendix C shows a 42-item test, but question ownership, scoring and version control are not detailed.	Version the questionnaire; require psychometrician approval before changes; preserve scoring rules and consent.

1.3 Recommended decision register

ID	Decision area	Recommended decision
D-01	Frontend	React single-page application built with Vite; responsive web, not native mobile.
D-02	Backend	Laravel REST API to preserve alignment with the proposal and provide validation, authentication, policies and reporting.
D-03	Database	PostgreSQL as the canonical database; confirm Hostinger plan support before final commitment.
D-04	Recommendation approach	Hybrid design: explainable RIASEC + admission-rule ranking for MVP, optional Decision Tree/Random Forest as a validated enhancement.
D-05	Official exam data	Only authorized staff can create or verify admission exam results; all edits are audited.
D-06	RIASEC scoring	Questionnaire is versioned. The original 42-item structure is retained unless the psychometrician formally approves changes.
D-07	Outputs	Top 3-5 recommendations, eligibility status, explanation, course requirements, and printable recommendation report.
D-08	Deployment	Separate development, staging and production environments; production hosted with HTTPS, backups and monitoring.

2. Recommended Scope and Success Criteria

2.1 Minimum Viable Product (MVP)

- Secure registration, login, logout, password reset, role-based access, and account status controls.
- Student profile and application record for one admission cycle.
- Staff encoding or CSV import of admission examination results, with verification status and audit log.
- Versioned 42-item RIASEC questionnaire, autosave, submission lock, scoring and top-three Holland code.
- Course catalog with descriptions, board-regulated flag, requirements, active status and admission cycle.
- Configurable RIASEC weights and admission thresholds per course.
- Explainable recommendation run that stores input snapshot, ranks courses and identifies ineligible choices.
- Student recommendation page with course details, reasons, guidance notes and accept/reject/undecided response.
- Administrative dashboards, applicant search/filter, recommendation review and printable reports.
- Audit logs, backups, error logging, basic performance monitoring and tested deployment.

2.2 Phase 2 enhancements

- Psychometrician-reviewed training dataset and comparison of Decision Tree versus Random Forest.
- Model versioning, metrics, approval and rollback; model prediction never bypasses eligibility rules.
- Email notifications, appointment scheduling, and guidance counselor notes.
- Course-capacity awareness and integration with existing admission/enrollment systems.
- Long-term outcome tracking for retention, shifting, grades and graduation, subject to expanded study scope.
- Progressive Web App features such as installability and limited offline questionnaire recovery.

2.3 Explicitly out of scope for the MVP

Out-of-scope item	Reason
Automatic enrollment or final course assignment	The proposal defines the platform as decision support; the student retains the final choice.
External schools and courses	The proposal limits recommendations to programs currently offered by TCC.

Out-of-scope item	Reason
Diagnosis or psychological treatment	RIASEC is used only for vocational-interest guidance.
Native Android/iOS app	Responsive React web is sufficient for the stated web-based scope.
Unvalidated generative-AI recommendations	Recommendations must be reproducible, auditable and policy-based.
Long-term academic outcome prediction	The proposal explicitly excludes long-term tracking.

2.4 Recommended measurable success criteria

Category	Acceptance target
Functional completeness	100% of approved MVP user stories implemented; at least 95% test cases passing before UAT.
Recommendation reproducibility	The same verified inputs and algorithm version always produce the same ranked result.
Expert agreement	Top-three recommendations reviewed against psychometrician-approved cases; team sets a target after pilot data is available.
Usability	System Usability Scale administered to intended users; project target of 70 or higher unless adviser sets another threshold.
Performance	Normal dashboard/API requests complete within 2 seconds at the 95th percentile in the staging load test; recommendation generation within 3 seconds.
Reliability	No open critical defects at launch; failed submissions are recoverable without losing completed answers.
Security	No student can access another student's data; all privileged changes are authorized and logged.
Operational readiness	Successful backup restore test, deployment checklist, user manual and administrator handover.

3. Stakeholders, Roles, and Governance

3.1 User roles

Role	Primary permissions	Important restrictions
Student applicant	Register; complete profile; take RIASEC assessment; view results and recommendations; record decision; download own report.	Cannot create/verify official exam scores or change course rules.
Guidance/Psychometrician	Review profiles and RIASEC results; validate questions, weights and interpretation text; review recommendations; generate guidance reports.	Should approve any scoring or instrument change.
Admission/Testing staff	Create/import/verify admission exam results; search applicants; correct data with reason.	Cannot alter psychometric mappings unless separately authorized.
System administrator	Manage users, roles, admission cycles, courses, settings, backups and audit review.	Should not silently alter student results; every privileged action is logged.
Capstone developer/maintainer	Deploy, troubleshoot, migrate database and support users through documented procedures.	No production data access beyond approved support needs.

3.2 RACI ownership matrix

Work item	Guidance/Psychometrician	Admission/Testing	Capstone team	TCC IT/Admin
Requirements approval	A	R	C	C
RIASEC questionnaire/scoring	A/R	C	C	I
Course cutoffs and policies	C	A/R	C	I
React UI and accessibility	C	C	A/R	I
API and database	C	C	A/R	I
Recommendation algorithm	A	C	R	C
Test-case acceptance	A/R	R	R	I
Production deployment	C	C	R	A

R = Responsible, A = Accountable, C = Consulted, I = Informed. Final assignments require confirmation by TCC.

3.3 Approval gates

1. Requirements baseline: user roles, workflow, official score format, course list, board-course threshold and report format approved.
2. Design baseline: use cases, DFD, ERD, architecture, wireframes and API contract approved.
3. Algorithm validation: RIASEC scoring, course weights, eligibility logic and sample outputs signed off by the psychometrician.
4. UAT release: no critical defects, security checks passed, SUS instrument ready and test data anonymized.
5. Production launch: backup/restore tested, domain/HTTPS configured, user accounts prepared and operating procedure accepted.

4. Functional Requirements and Modules

ID / Module	Required capabilities	Users
FR-01 Authentication and access	Registration, email verification if available, login, logout, password reset, role-based route/API protection, inactive/locked accounts.	Student, all staff
FR-02 Student profile	Personal and application details, SHS information, contact data, completeness status, editable fields and change history.	Student, authorized staff
FR-03 Admission exam processing	Manual encoding and CSV import, format validation, duplicate checking, verification, correction reason and score history.	Testing/Admission
FR-04 RIASEC questionnaire management	Questionnaire version, ordered questions, R/I/A/S/E/C category, instructions, active dates and approval status.	Psychometrician/Admin
FR-05 Assessment taking	Consent/instructions, progress, autosave, resume, final confirmation, single completed session per policy, accessibility.	Student
FR-06 RIASEC scoring	Category totals, normalized scores, ranked categories, tie handling, Holland code and interpretation.	System/Guidance
FR-07 Course catalog	Course code/name, department, description, board-regulated flag, active cycle, requirements and guidance content.	Admin/Guidance
FR-08 Course fit rules	RIASEC weights, admission threshold, rule effective dates, active status and version history.	Guidance/Admission
FR-09 Recommendation engine	Eligibility filtering, interest similarity, academic fit, composite score, top ranking, explanation and algorithm version.	System

ID / Module	Required capabilities	Users
FR-10 Recommendation review	Student view, guidance review, accept/reject/undecided decision, optional reason and printable report.	Student/Guidance
FR-11 Dashboards and search	Applicant counts, assessment status, recommendations, filters, pagination and export.	Staff
FR-12 Reports	Individual recommendation report, applicant status list, RIASEC distribution, course recommendation summary and CSV/PDF export.	Guidance/Admin
FR-13 Audit and settings	Privileged action logs, system settings, admission cycles, algorithm version and error logs.	Admin

4.1 Authorization matrix

Action	Student	Guidance	Admission/Testing	System Admin
View own profile/result	Yes	No	No	No
Edit own profile	Yes	No	No	No
View all applicants	No	Yes	Yes	Yes
Encode/verify exam score	No	Read	Create/Edit	Configure
Take RIASEC assessment	Yes	No	No	No
Edit RIASEC questions	No	Approve/Edit	No	Manage access
Edit course weights	No	Approve/Edit	Consult	Manage access
Run recommendation	Own when complete	Any applicant	Any applicant	Any applicant
Generate reports	Own	All	Operational	All
Manage users and roles	No	No	No	Yes
View audit logs	No	Limited	Limited	Yes

4.2 Important validation rules

- One unique applicant number and application record per admission cycle.
- A recommendation cannot be generated until the profile is complete, the exam result is verified, and the RIASEC assessment is completed.
- A completed assessment is immutable. Corrections require a new session or an authorized reset with reason.
- Every question in an active questionnaire must have one valid RIASEC dimension and a unique sequence number.
- Course RIASEC weights must be non-negative and normalized; the system warns if the total is not 1.0.
- Course admission rules must have an effective admission cycle and cannot be retroactively changed without creating a new version.
- Only active courses are included in new recommendation runs; historical runs preserve the original input snapshot.
- Rank values are unique within a recommendation run. Final scores are stored with algorithm and model versions.
- Student decisions do not overwrite the generated ranking; they are separate records.

5. Business Workflow and Rules

5.1 End-to-end student flow

Figure 1. Recommended end-to-end workflow from registration to guidance report.

5.2 Application state model

State	Meaning	Allowed transition
DRAFT	Student registered but profile incomplete.	Profile completed -> PROFILE_COMPLETE
PROFILE_COMPLETE	Required student information is valid.	Exam result encoded -> EXAM_PENDING_VERIFICATION
EXAM_PENDING_VERIFICATION	Exam result exists but is not official.	Staff verifies -> EXAM_VERIFIED
EXAM_VERIFIED	Official admission result is available.	Assessment starts -> ASSESSMENT_IN_PROGRESS
ASSESSMENT_IN_PROGRESS	Answers are autosaved but not submitted.	Submit all answers -> ASSESSMENT_COMPLETED
ASSESSMENT_COMPLETED	RIASEC scores are final for this session.	Generate -> RECOMMENDED
RECOMMENDED	A ranked recommendation run is available.	Student action -> DECIDED or remains RECOMMENDED
DECIDED	Student recorded accept/reject/other choice.	Guidance report finalized -> CLOSED
CLOSED	Cycle record is complete and read-only except authorized correction.	New cycle creates a new application.

5.3 Course eligibility rule

Policy confirmation required
The proposal states that board-regulated courses require an admission grade of 2.50 or better. Before coding, confirm that lower values are better, whether the threshold applies to all board courses, and whether each program has additional requirements.

Recommended implementation: store course-specific eligibility rules rather than hard-coding a single threshold. Each rule should identify the admission cycle, comparison direction, threshold, board-course flag, and any approved notes. Eligibility is evaluated before ranking so an academically ineligible course is never presented as an unrestricted recommendation.

5.4 Data ownership and correction workflow

- Student-created data: account, contact details, profile responses and assessment answers.
- Staff-controlled official data: applicant number, admission exam result, verification status, course rules and questionnaire publication.
- System-generated data: RIASEC scores, recommendation runs, rank explanations, reports and audit logs.
- Correction rule: official data edits require an authorized user, reason, previous value, new value, timestamp and affected recommendation regeneration decision.

6. Recommended Technical Architecture

6.1 Layered architecture

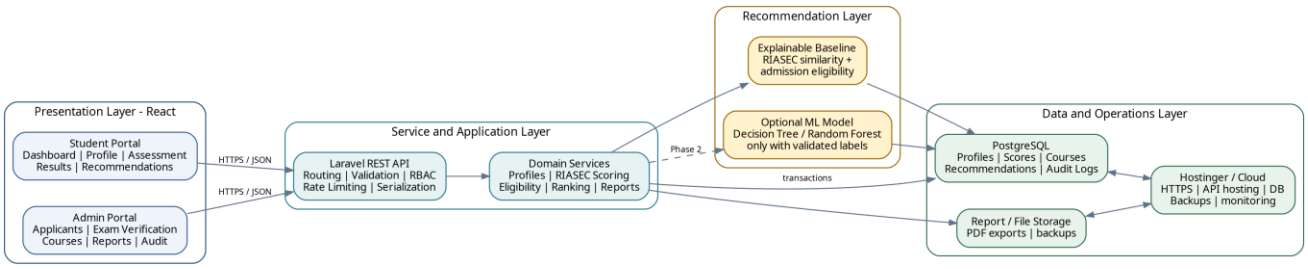

Figure 2. Recommended React-based architecture aligned with the proposal's layered design.

6.2 Technology stack

Layer	Recommended tools	Reason
Frontend	React + Vite, React Router, TanStack Query, React Hook Form, schema validation library, accessible component system	Fast development, reusable pages, strong form/data handling.
Backend API	Laravel REST API with request validation, policies, jobs, mail and PDF/report service	Matches proposal while separating frontend from backend.
Authentication	Secure session cookie or token approach; same-site cookies preferred when frontend/API share the same parent domain	Reduces token exposure; supports role-based access.
Database	PostgreSQL	Strong relational constraints, JSON support for snapshots/metrics and reliable indexing.
Recommendation baseline	Laravel service using deterministic scoring and configurable weights	Transparent, testable and usable without a large training dataset.
Optional ML	Python training notebook/service using Decision Tree and Random Forest	Only after validated labeled data exists.
Reports	Server-side PDF generation plus CSV export	Consistent printable guidance documents.
Deployment	Hostinger or equivalent: React static build, Laravel API, managed database, HTTPS, backups	Aligns with the proposal; exact hosting capabilities must be confirmed.
Source control/CI	GitHub or GitLab with protected main branch, pull requests, automated tests and build checks	Supports five-person team collaboration and reproducibility.

6.3 Suggested repository structure

```
capstone-recommendation-system/
  apps/
    web/                  # React frontend
      src/
        api/              # typed API client
        components/       # shared UI
        features/         # auth, profile, assessment, recommendation
        layouts/          # student/admin layouts
        pages/            # route-level pages
        routes/           # route definitions and guards
        schemas/          # form validation schemas
        hooks/            # reusable hooks
        utils/
    api/                  # Laravel REST API
      app/
        Http/Controllers/Api/
```

```
    Http/Requests/
    Models/
    Policies/
    Services/
      RiasecScoringService.php
      EligibilityService.php
      RecommendationService.php
      ReportService.php
    Jobs/
  database/
    migrations/
    seeders/
    factories/
  routes/api.php
  tests/
ml/                    # optional Phase 2 experiments
  notebooks/
  training/
  models/
  model-card.md
docs/
  requirements/
  diagrams/
  api/
  test-cases/
.github/workflows/
README.md
```

6.4 Environment plan

Environment	Data and controls	Purpose
Local development	Each member's machine; seeded fake data; local database; never use production data.	Feature development and unit tests
Shared development	Optional team server or branch preview.	Integration before merging
Staging/UAT	Production-like configuration with anonymized pilot data.	Scenario testing, SUS and client approval
Production	Restricted credentials, HTTPS, scheduled backups, monitoring and audit logs.	Institutional use

6.5 Non-functional requirements

Quality attribute	Required approach
Security	Server-side authorization for every protected endpoint; password hashing; validation; CSRF/CORS configuration; rate limiting; audit logs.
Availability	Production target agreed with TCC; graceful maintenance page; daily backup and tested restore.
Performance	Pagination for applicant lists; indexed search; no loading all 8,000 applicants in one response; background jobs for reports/imports.
Reliability	Database transactions for assessment submission and recommendation generation; idempotent imports; retryable jobs.
Usability	Mobile-responsive, clear progress, plain-language explanations, keyboard support and readable contrast.
Maintainability	Layered services, migrations, seeders, API documentation, tests and coding standards.
Traceability	Store algorithm version, input snapshot, rule version and user actions for every recommendation.

7. Database Design and Data Dictionary

7.1 Database choice and consistency rule

The paper is inconsistent: one section describes MySQL/phpMyAdmin, while the architecture diagram specifies PostgreSQL. The team must select one database before migrations begin. This roadmap uses PostgreSQL because it supports strong relational constraints and JSONB fields for recommendation snapshots and model metrics. If the chosen Hostinger plan or team skill set requires MySQL, use the same logical schema but replace PostgreSQL-specific types and update every document and diagram.

7.2 High-level ERD

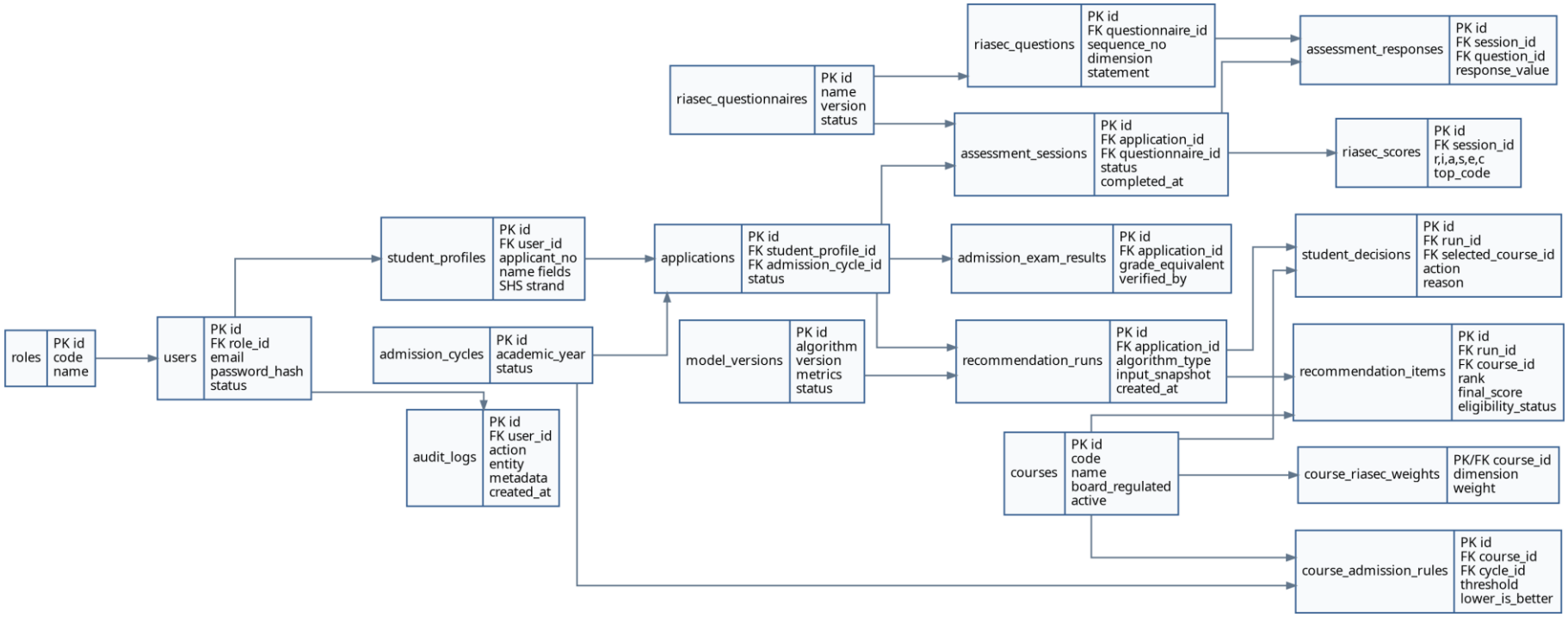

Figure 3. High-level relational design. Detailed columns and constraints follow.

7.3 Core data dictionary

roles

Column	Type	Constraint	Purpose
id	SMALLINT	PK	Role identifier.
code	VARCHAR(30)	UNIQUE, NOT NULL	student, guidance, admission, admin.
name	VARCHAR(80)	NOT NULL	Display name.

users

Column	Type	Constraint	Purpose
id	UUID	PK	Account identifier.
role_id	SMALLINT	FK roles	Primary role.
email	VARCHAR(255)	UNIQUE, NOT NULL	Normalized login email.
password_hash	VARCHAR(255)	NOT NULL	Framework-managed password hash.
status	VARCHAR(20)	NOT NULL	pending, active, locked, inactive.
last_login_at	TIMESTAMPTZ	NULL	Last successful login.
created_at / updated_at	TIMESTAMPTZ	NOT NULL	Audit timestamps.

student_profiles

Column	Type	Constraint	Purpose
id	UUID	PK	Student profile ID.
user_id	UUID	FK users, UNIQUE	One profile per student account.
applicant_no	VARCHAR(40)	UNIQUE	Official applicant number.
first_name / middle_name / last_name	VARCHAR	NOT NULL where required	Applicant name.
birth_date	DATE	NULL/required by policy	Only collect if institution requires it.
contact_no	VARCHAR(30)	NULL	Applicant contact.
address_json	JSONB	NULL	Structured address without excessive columns.
shs_strand	VARCHAR(60)	NULL	Optional approved profiling field.
school_name	VARCHAR(180)	NULL	Previous school.
consent_at	TIMESTAMPTZ	NULL	Assessment/privacy acknowledgement.

admission_cycles

Column	Type	Constraint	Purpose
id	UUID	PK	Admission period ID.
name	VARCHAR(80)	NOT NULL	Example: AY 2026-2027 First Semester.
academic_year	VARCHAR(20)	NOT NULL	Reporting field.
starts_at / ends_at	DATE	NOT NULL	Cycle window.
status	VARCHAR(20)	NOT NULL	draft, open, closed, archived.

applications

Column	Type	Constraint	Purpose
id	UUID	PK	Applicant-cycle record.
student_profile_id	UUID	FK student_profiles	Applicant.
admission_cycle_id	UUID	FK admission_cycles	Cycle.
application_no	VARCHAR(50)	UNIQUE	Official application reference.
preferred_course_id	UUID	FK courses, NULL	Initial preference, not recommendation.
status	VARCHAR(40)	NOT NULL	Application state model.
submitted_at	TIMESTAMPTZ	NULL	Profile submission time.
UNIQUE	(student_profile_id, admission_cycle_id)	Constraint	Prevents duplicate application per cycle.

admission_exam_results

Column	Type	Constraint	Purpose
id	UUID	PK	Exam result record.
application_id	UUID	FK applications, UNIQUE	One current official result per application.
raw_score	NUMERIC	NULL	Optional raw institutional score.
grade_equivalent	NUMERIC(5,2)	NOT NULL	Value used by eligibility rules.
source	VARCHAR(30)	NOT NULL	manual, csv_import, integration.
verification_status	VARCHAR(20)	NOT NULL	pending, verified, rejected.
verified_by	UUID	FK users, NULL	Authorized staff.
verified_at	TIMESTAMPTZ	NULL	Verification time.
remarks	TEXT	NULL	Correction/validation notes.

riasec_questionnaires

Column	Type	Constraint	Purpose
id	UUID	PK	Questionnaire version.
name	VARCHAR(120)	NOT NULL	Instrument name.
version	VARCHAR(30)	UNIQUE, NOT NULL	Example: TCC-RIASEC-1.0.
instructions	TEXT	NOT NULL	Student instructions.
status	VARCHAR(20)	NOT NULL	draft, approved, active, retired.
approved_by	UUID	FK users, NULL	Psychometrician approval.
approved_at	TIMESTAMPTZ	NULL	Approval timestamp.

riasec_questions

Column	Type	Constraint	Purpose
id	UUID	PK	Question ID.

Column	Type	Constraint	Purpose
questionnaire_id	UUID	FK questionnaires	Questionnaire owner.
sequence_no	SMALLINT	NOT NULL	Display order; unique per version.
statement	TEXT	NOT NULL	RIASEC statement.
dimension	CHAR(1)	CHECK R/I/A/S/E/C	Scoring category.
reverse_scored	BOOLEAN	DEFAULT FALSE	Usually false for the shown binary instrument.
is_active	BOOLEAN	DEFAULT TRUE	Publication control.

assessment_sessions

Column	Type	Constraint	Purpose
id	UUID	PK	Student attempt/session.
application_id	UUID	FK applications	Applicant-cycle context.
questionnaire_id	UUID	FK questionnaires	Exact version used.
status	VARCHAR(20)	NOT NULL	in_progress, completed, void.
started_at / completed_at	TIMESTAMPTZ	NULL	Timing data.
attempt_no	SMALLINT	DEFAULT 1	Controlled retakes.
submitted_ip	INET	NULL	Security/audit support.

assessment_responses

Column	Type	Constraint	Purpose
id	UUID	PK	Response ID.
session_id	UUID	FK assessment_sessions	Attempt.
question_id	UUID	FK riasec_questions	Question.
response_value	SMALLINT	CHECK approved scale	For binary instrument: 0 or 1.
answered_at	TIMESTAMPTZ	NOT NULL	Autosave/submission time.
UNIQUE	(session_id, question_id)	Constraint	One answer per question per attempt.

riasec_scores

Column	Type	Constraint	Purpose
id	UUID	PK	Calculated score record.
session_id	UUID	FK sessions, UNIQUE	One final score per completed session.
r_score ... c_score	SMALLINT	NOT NULL	Six dimension totals.
top_code	VARCHAR(6)	NOT NULL	Top categories, including tie notation policy.
normalized_scores	JSONB	NOT NULL	0-1 values used by ranking.
scoring_version	VARCHAR(30)	NOT NULL	Reproducibility.
calculated_at	TIMESTAMPTZ	NOT NULL	Calculation time.

courses

Column	Type	Constraint	Purpose
id	UUID	PK	Course/program ID.
code	VARCHAR(30)	UNIQUE, NOT NULL	Institutional course code.
name	VARCHAR(180)	NOT NULL	Official program name.
department	VARCHAR(120)	NULL	Reporting grouping.
description	TEXT	NULL	Student-facing summary.
board_regulated	BOOLEAN	DEFAULT FALSE	Policy flag.
requirements_text	TEXT	NULL	Guides and requirements.
is_active	BOOLEAN	DEFAULT TRUE	Recommendation availability.

course_riasec_weights

Column	Type	Constraint	Purpose
id	UUID	PK	Weight row.
course_id	UUID	FK courses	Course profile.
dimension	CHAR(1)	CHECK R/I/A/S/E/C	RIASEC category.
weight	NUMERIC(6,5)	CHECK 0-1	Course preference weight.
version	VARCHAR(30)	NOT NULL	Mapping version.
UNIQUE	(course_id, dimension, version)	Constraint	One weight per dimension/version.

course_admission_rules

Column	Type	Constraint	Purpose
id	UUID	PK	Rule ID.
course_id	UUID	FK courses	Target course.
admission_cycle_id	UUID	FK cycles	Effective period.
score_field	VARCHAR(30)	NOT NULL	Example: grade_equivalent.
operator	VARCHAR(5)	CHECK approved values	<=, >=, <, >, between.
threshold_value	NUMERIC(8,2)	NOT NULL	Approved cutoff.
is_mandatory	BOOLEAN	DEFAULT TRUE	Gate or preference.
notes	TEXT	NULL	Policy explanation.

recommendation_runs

Column	Type	Constraint	Purpose
id	UUID	PK	Immutable run header.
application_id	UUID	FK applications	Applicant context.
algorithm_type	VARCHAR(30)	NOT NULL	rules, decision_tree, random_forest, hybrid.
algorithm_version	VARCHAR(40)	NOT NULL	Version identifier.
model_version_id	UUID	FK model_versions, NULL	ML model used, if any.

Column	Type	Constraint	Purpose
input_snapshot	JSONB	NOT NULL	Scores, rules and course versions used.
created_by	UUID	FK users, NULL	User/system initiator.
created_at	TIMESTAMPTZ	NOT NULL	Run time.

recommendation_items

Column	Type	Constraint	Purpose
id	UUID	PK	Ranked result row.
run_id	UUID	FK recommendation_runs	Run header.
course_id	UUID	FK courses	Recommended course.
rank	SMALLINT	NOT NULL	1 is highest.
interest_score	NUMERIC(6,3)	NOT NULL	0-100.
academic_score	NUMERIC(6,3)	NOT NULL	0-100.
final_score	NUMERIC(6,3)	NOT NULL	Composite.
eligibility_status	VARCHAR(20)	NOT NULL	eligible, conditional, ineligible.
explanation	JSONB	NOT NULL	Human-readable factors.
UNIQUE	(run_id, rank)	Constraint	No duplicate rank.

student_decisions

Column	Type	Constraint	Purpose
id	UUID	PK	Decision record.
run_id	UUID	FK runs	Recommendation considered.
selected_course_id	UUID	FK courses, NULL	Student selection.
action	VARCHAR(20)	NOT NULL	accepted, rejected, undecided, chose_other.
reason	TEXT	NULL	Optional feedback.
decided_at	TIMESTAMPTZ	NOT NULL	Decision time.

model_versions

Column	Type	Constraint	Purpose
id	UUID	PK	ML model registry.
name / version	VARCHAR	NOT NULL	Model identity.
algorithm	VARCHAR(40)	NOT NULL	Decision Tree or Random Forest.
training_dataset_ref	TEXT	NOT NULL	Dataset provenance.
feature_schema	JSONB	NOT NULL	Input columns and preprocessing.
metrics	JSONB	NOT NULL	Accuracy, macro-F1, top-k metrics.
status	VARCHAR(20)	NOT NULL	draft, validated, active, retired.
approved_by / approved_at	UUID/TIMESTAMPTZ	NULL	Governance.

audit_logs

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Append-only event.
user_id	UUID	FK users, NULL	Actor.
action	VARCHAR(100)	NOT NULL	Example: exam_result.updated.
entity_type / entity_id	VARCHAR / UUID	NULL	Affected record.
old_values / new_values	JSONB	NULL	Change details, excluding secrets.
ip_address	INET	NULL	Security context.
created_at	TIMESTAMPTZ	NOT NULL	Event time.

7.4 Indexes, constraints, and data integrity

- Unique indexes: users.email; student_profiles.applicant_no; applications.application_no; questionnaire version; course code.
- Composite indexes: applications(admission_cycle_id, status); assessment_sessions(application_id, status); recommendation_items(run_id, rank); audit_logs(entity_type, entity_id, created_at).
- Search indexes: normalized applicant name, applicant number and email. Use database-supported case-insensitive search where appropriate.
- Foreign keys use RESTRICT for reference/master data and carefully selected CASCADE only for child rows that have no independent meaning.
- Completed assessments and recommendation runs are immutable; corrections produce a new version or run.
- All timestamp fields use timezone-aware values. The application displays them in the institution's local timezone.
- Database transactions wrap assessment submission, score creation, recommendation generation and imported batches.

7.5 Seed data required before development can finish

Seed dataset	Required content
Roles and permissions	Approved role list and authorization matrix.
Admission cycle	Current academic year/term and application dates.
RIASEC questionnaire	Final 42 statements, dimension mapping, scoring scale, instructions and version.
Course catalog	Official TCC programs, codes, descriptions, board status, requirements and active status.
Course RIASEC profiles	Psychometrician-approved weights or top Holland codes for each course.
Admission rules	Verified cutoff/comparison rule per course and cycle.
Test users	Anonymized student and staff accounts for staging.
Validation cases	At least 30-50 expert-reviewed sample profiles with expected top recommendations for algorithm testing.

8. Recommendation Engine Design

8.1 Recommended MVP: explainable hybrid rules

The proposal names machine learning, but it does not define a valid training label or historical outcome dataset. Because the study also excludes long-term tracking, the system cannot honestly learn "successful course fit" from outcomes yet. Therefore, the production MVP should use a transparent expert-configured ranking model. An ML

experiment may be added for the capstone objective, but it should be compared against the rules baseline and clearly labeled as experimental until validated.

Do not fake machine learning
A Decision Tree or Random Forest trained on arbitrary or invented labels may produce impressive-looking outputs but has no defensible validity. Use psychometrician-approved labeled cases, document the dataset source, and retain a deterministic baseline.

8.2 RIASEC scoring

1. The Appendix C instrument contains 42 statements and the six categories Realistic, Investigative, Artistic, Social, Enterprising and Conventional.
2. Preserve the approved response scale. If the instrument is binary, store 0/1; do not convert it to a Likert scale without psychometrician approval.
3. For each category, sum the response values of questions mapped to that category.
4. Normalize each category by the maximum possible score for that questionnaire version.
5. Sort category scores to produce the top-three Holland code. Define a transparent tie rule, and show tied dimensions instead of hiding them.

```
For student vector S = [R, I, A, S, E, C]
normalized category score:
    s_d = raw_score_d / maximum_possible_score_d

For course profile W_c = [w_R, w_I, w_A, w_S, w_E, w_C]
where each weight is between 0 and 1 and sum(W_c) = 1.

Interest fit (simple weighted fit):
    interest_fit_c = 100 * sum(s_d * w_c,d)

Alternative shape similarity (recommended for comparison):
    cosine_fit_c = 100 * cosine_similarity(S, W_c)
```

8.3 Admission eligibility and academic fit

Eligibility must be evaluated using the official verified examination result and the course rule effective for the applicant's admission cycle. For a confirmed "2.50 or better" lower-is-better policy, a board course is eligible when grade_equivalent <= 2.50. The comparison direction must be stored in the rule, not assumed globally.

```
if mandatory eligibility rule fails:
    eligibility_status = "ineligible"
    academic_fit = 0
else:
    eligibility_status = "eligible" or "conditional"
    academic_fit = policy-defined value from 0 to 100

Example continuous academic fit for lower-is-better grades:
    margin = threshold - student_grade
    academic_fit = clamp(50 + margin * scale_factor, 0, 100)

The scale factor must be approved and tested. A simpler MVP may use
academic_fit = 100 for eligible and 0 for ineligible.
```

8.4 Composite ranking and explanation

```
Recommended starting weights (subject to client validation):
    final_score = 0.70 * interest_fit + 0.30 * academic_fit

Hard rule:
    Ineligible courses are not shown in the unrestricted top list.
```

They may appear in a separate "not currently eligible" section with reason.

Tie-break order:
1. higher interest fit
2. stronger academic margin
3. official course code for deterministic ordering

Each recommendation item should explain: the student's top RIASEC categories, the course's matching categories, admission-rule result, score components, applicable requirements, and a disclaimer that the output supports guidance rather than replacing the student's decision.

8.5 Illustrative example only

Item	Illustrative value
Student normalized RIASEC	R 0.57, I 0.86, A 0.29, S 0.71, E 0.43, C 0.57
Verified admission grade	2.25; assumed lower-is-better for illustration
Illustrative Course X profile	R 0.10, I 0.35, A 0.10, S 0.15, E 0.10, C 0.20
Interest fit	Weighted fit calculated from the six normalized scores
Eligibility	Eligible if Course X rule is <= 2.50
Explanation	Strong Investigative and Social alignment; verified score meets the cutoff; specific course requirements still apply.

The example is not an official TCC course mapping. Actual mappings and weights must be approved by the Guidance Office/psychometrician.

8.6 Machine-learning enhancement plan

Step	Required action
Define label	Preferred target: psychometrician-approved suitable course set or validated historical outcome. "Course chosen" alone is not necessarily suitability.
Define features	RIASEC scores, verified admission score and only approved profile variables. Avoid unnecessary sensitive attributes.
Prepare dataset	Document source, consent, missing data, class balance, anonymization and sample size.
Split correctly	Keep applicant records separated; use train/validation/test or cross-validation. Prevent duplicate applicant leakage.
Train baselines	Majority-class baseline, explainable rules, Decision Tree and Random Forest.
Evaluate	Macro-F1, per-class recall, top-3 recall, confusion matrix, calibration and expert review. Accuracy alone is insufficient.
Approve model	Create model card, metrics, dataset reference, known limitations and psychometrician sign-off.
Deploy safely	Eligibility rules remain authoritative. Store model version and prediction. Provide rollback to rules baseline.
Monitor	Track disagreement, user feedback, performance drift and class imbalance; retrain only through an approved process.

8.7 Recommendation validation dataset

Field	Purpose
case_id	Anonymous test identifier
r_score..c_score	Six category values
admission_grade	Verified or synthetic-but-policy-valid test value

Field	Purpose
eligible_courses	Expected eligibility set
expected_top_courses	Psychometrician-approved top 3 or acceptable set
rationale	Why each expected course fits
reviewer	Authorized reviewer code
approval_date	Validation timestamp
dataset_version	Used by tests/model card

9. REST API and Integration Plan

9.1 API principles

- All authorization is enforced on the backend; hiding a React button is not security.
- Use versioned endpoints such as /api/v1 and consistent JSON response/error formats.
- Use request validation classes and policy checks for every write operation.
- Paginate applicant, report and audit lists. Support filters without returning entire tables.
- Assessment response endpoints are idempotent so autosave retries do not duplicate answers.
- Recommendation generation is transactional and returns the immutable run identifier.
- Document endpoints using OpenAPI so React and backend developers share one contract.

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Create student account.
POST	/api/v1/auth/login	Authenticate user.
POST	/api/v1/auth/logout	End session.
POST	/api/v1/auth/forgot-password	Begin password reset.
GET	/api/v1/me	Current user and permissions.
GET/PATCH	/api/v1/student/profile	Read/update own student profile.
POST	/api/v1/applications	Create current-cycle application.
GET	/api/v1/applications/{id}	Read authorized application.
POST	/api/v1/applications/{id}/exam-result	Staff encodes official result.
POST	/api/v1/applications/{id}/exam-result/verify	Staff verifies result.
POST	/api/v1/exam-results/import	CSV import with validation preview.
GET	/api/v1/riasec/questionnaire/active	Get published questionnaire.
POST	/api/v1/assessment-sessions	Start/resume assessment.
PUT	/api/v1/assessment-sessions/{id}/responses/{questionId}	Autosave one answer.
POST	/api/v1/assessment-sessions/{id}/submit	Validate and finalize assessment.
GET	/api/v1/applications/{id}/riasec-result	Read RIASEC result.
POST	/api/v1/applications/{id}/recommendations	Generate a new run.
GET	/api/v1/applications/{id}/recommendations/latest	Get latest run.
GET	/api/v1/recommendation-runs/{id}	Get ranked items and explanation.
POST	/api/v1/recommendation-runs/{id}/decision	Record student decision.

Method	Endpoint	Purpose
GET/POST/PATCH	/api/v1/admin/courses	Manage course catalog.
GET/POST/PATCH	/api/v1/admin/course-rules	Manage cutoffs and RIASEC weights.
GET/POST/PATCH	/api/v1/admin/questionnaires	Manage questionnaire versions.
GET	/api/v1/admin/applicants	Search/filter applicants.
GET	/api/v1/admin/reports/...	Generate operational reports.
GET	/api/v1/admin/audit-logs	Review privileged actions.

9.2 Standard API response shape

```
Success:
{
  "data": { ... },
  "meta": { "requestId": "...", "version": "v1" }
}

Validation error:
{
  "message": "The given data was invalid.",
  "errors": {
    "grade_equivalent": ["A valid verified grade is required."]
  },
  "requestId": "..."
}
```

9.3 CSV import flow

1. Upload file to a temporary location.
2. Parse headers and show a preview; do not immediately write to official records.
3. Validate applicant number, numeric score, duplicates and admission cycle.
4. Display accepted rows, rejected rows and reasons.
5. Authorized user confirms import.
6. Write in a transaction or background job, log the batch and provide a result file.
7. Do not overwrite verified values without an explicit correction workflow.

10. React UI/UX Plan

10.1 Student screen inventory

Screen	Purpose
S-01 Login/Register	Simple account access, validation, password reset.
S-02 Onboarding/Profile	Stepper for identity, contact, SHS and application details; save draft.
S-03 Student Dashboard	Completion checklist: profile, exam verification, assessment, recommendation.
S-04 Assessment Instructions	Purpose, consent, no-wrong-answer statement, time expectation and privacy note.
S-05 RIASEC Assessment	Progress bar, question groups, autosave, previous/next, keyboard-friendly controls.
S-06 Submission Review	Unanswered-question warning and final confirmation.

Screen	Purpose
S-07 RIASEC Result	Six scores, top code, plain-language interpretation and disclaimer.
S-08 Recommendations	Top ranked course cards, fit score, eligibility badge and why it fits.
S-09 Course Detail	Description, matched traits, academic rule, requirements and guidance notes.
S-10 Decision/Report	Accept/reject/undecided response and downloadable report.

10.2 Administrative screen inventory

Screen	Purpose
A-01 Dashboard	Counts by application state, completion status and recommendation distribution.
A-02 Applicants	Search, filters, pagination, bulk actions and CSV export.
A-03 Applicant Detail	Profile, exam history, assessment result, recommendations, decision and audit trail.
A-04 Exam Import/Verification	Upload preview, row validation, verification and corrections.
A-05 Questionnaire Manager	Version, questions, category mapping, preview, approval and publication.
A-06 Course Catalog	Course information, board flag, active status and guidance content.
A-07 Rules and Weights	Admission cutoff and six RIASEC weights with validation/version history.
A-08 Recommendation Review	Re-run, compare versions and document guidance notes.
A-09 Reports	Individual and aggregate reports; PDF/CSV generation.
A-10 Users/Audit/Settings	Accounts, permissions, admission cycles, logs and system configuration.

10.3 React implementation conventions

- Use feature folders so assessment, profile and recommendation logic remain isolated and testable.
- Use TanStack Query for server state and caching; avoid copying API data into a large global store.
- Use React Hook Form plus shared validation schemas for complex forms and consistent error messages.
- Protect routes using current-user permissions, but still rely on backend policy checks.
- Use error boundaries and a standard empty/loading/error state for every data page.
- Use a design system with consistent buttons, forms, cards, badges, tables, dialogs and notifications.
- Add accessibility labels, visible focus, keyboard navigation, responsive tables and readable chart alternatives.
- Never communicate course suitability only by color; include text labels such as Eligible, Conditional or Not Eligible.

10.4 Suggested component map

```
AppShell
  AuthGuard / PermissionGuard
  StudentLayout
    CompletionChecklist
    ProfileForm
    AssessmentProgress
    RiasecScoreSummary
    RecommendationCard
    CourseFitExplanation
    DecisionForm
  AdminLayout
    ApplicantTable
    ApplicantStatusFilters
```

```
    ExamResultEditor
    CsvImportWizard
    QuestionnaireEditor
    CourseRuleEditor
    ReportFilters
    AuditLogTable
  Shared
    PageHeader
    FormField
    ConfirmDialog
    DataTable
    StatusBadge
    EmptyState
    ErrorState
    LoadingSkeleton
```

10.5 UX rules for the assessment

- Show one concise instruction set before the first item; avoid coaching students toward a category.
- Display progress accurately and allow resume before submission.
- Autosave after each response and show a subtle saved status.
- Do not reveal category scoring while the assessment is in progress.
- Require all mandatory answers before submission and identify missing items.
- After submission, make answers read-only and show the questionnaire version used.
- Provide a text summary in addition to any radar/bar chart.

11. Security, Privacy, and Audit Controls

Control area	Required implementation
Authentication	Strong password policy, secure password hashing, session expiration, password reset, account lockout/rate limiting.
Authorization	Backend policies by role and ownership; students only access their own records.
Input protection	Server-side validation, output encoding, prepared ORM queries, upload type/size checks and safe CSV parsing.
Transport	HTTPS only in staging/production; secure cookie flags; correct CORS and CSRF settings.
Data minimization	Collect only information required for admission and recommendation. Avoid adding sensitive profile fields without written need.
Consent and transparency	Explain purpose, inputs, recommendation limitations, data use and contact for correction.
Auditability	Append-only logs for score verification, questionnaire publication, rule changes, recommendation runs and report generation.
Secrets	Environment variables and secret manager/hosting controls; never commit credentials or production .env files.
Backups	Encrypted or access-controlled backups, retention schedule, restore test and documented owner.
Logging	Do not log passwords, assessment answers in plain debug logs, access tokens or excessive personal data.

11.1 Audit events that must be captured

- User created, activated, locked, role changed or password reset by admin.
- Student profile official identifiers changed.
- Exam result created, imported, verified, rejected or corrected.
- Questionnaire approved, activated or retired.
- Course rule or RIASEC weight changed.
- Assessment reset or voided.
- Recommendation generated or regenerated, including algorithm version.

- Student decision recorded or changed.
- Report generated/downloaded by staff.
- Backup restore, deployment and configuration change.

11.2 Data retention questions for the client

1. How long should unsuccessful or un-enrolled applicant records remain in the system?
2. Who may export reports containing personal information?
3. When should accounts be archived or anonymized?
4. May anonymized assessment and decision data be used for future model training?
5. What is the official process for correction, deletion and data access requests?
6. Who owns backup access and incident notification?

12. Testing and Evaluation Strategy

12.1 Test layers

Layer	Coverage
Unit tests	RIASEC scoring, tie handling, eligibility comparison, final-score calculation, validators, permissions.
API feature tests	Authentication, profile ownership, assessment lifecycle, imports, recommendations, reports and audit events.
Frontend component tests	Forms, progress, validation, permissions, recommendation cards and error states.
Integration tests	React-to-API flow; database transactions; report generation; optional ML service.
End-to-end tests	Student registration through recommendation and staff import/verification through report.
Scenario-based testing	Realistic cases with expected input, output, actual result and pass/fail - directly aligned with the proposal.
Usability testing	Guidance/Admission staff and representative students; SUS questionnaire and task observation.
Performance testing	Applicant list/filter, simultaneous assessment autosaves, recommendation generation and report queues.
Security testing	Access-control attempts, IDOR checks, invalid uploads, rate limiting, session handling and dependency scan.
Recovery testing	Interrupted assessment, failed import, failed report job, backup restore and rollback.

12.2 Critical scenario-based test cases

ID	Scenario	Expected result
SBT-01	New student completes profile, verified exam and all assessment items.	RIASEC scores and top recommendations are generated once; report is available.
SBT-02	Student attempts recommendation before exam verification.	Blocked with a clear completion requirement; no run stored.
SBT-03	One assessment answer is missing at submission.	Submission rejected; missing question is identified; saved answers remain.
SBT-04	Board-course threshold is 2.50; student grade is 2.51 under lower-is-better rule.	Course marked ineligible and excluded from unrestricted top list.
SBT-05	Student grade is exactly 2.50.	Boundary behavior follows the approved <= rule.
SBT-06	Two RIASEC dimensions tie.	Tie is displayed according to documented policy; result is deterministic.
SBT-07	Staff changes a course weight after a recommendation was generated.	Old run remains unchanged; a new run uses the new version.

ID	Scenario	Expected result
SBT-08	CSV contains duplicate applicant numbers and invalid score.	Preview rejects invalid rows with reasons; valid rows are not duplicated.
SBT-09	Student changes URL to another applicant ID.	API returns unauthorized/forbidden and records security context as appropriate.
SBT-10	Recommendation service fails mid-transaction.	No partial run/items remain; user can retry safely.
SBT-11	100 simulated users autosave responses.	No lost answers; latency remains within target.
SBT-12	Backup is restored to staging.	Core records, relationships and latest report data are verified.

12.3 Recommendation-specific evaluation

- Create a fixed gold-standard case set approved by the psychometrician.
- Verify exact RIASEC scoring against manual calculation for every category.
- Verify course eligibility boundaries, including exactly-equal threshold values.
- Measure top-1 and top-3 agreement with the approved case set.
- Review explanations for correctness and consistency; a high score with a wrong explanation is a failure.
- For ML, compare against the deterministic baseline and report macro-F1, per-course recall and top-3 recall.
- Record dataset/model versions so results can be reproduced during defense.

12.4 Exit criteria before production

Area	Exit condition
Requirements	All MVP requirements traced to implemented story/test.
Defects	Zero critical/high-severity open defects; medium defects accepted by client or scheduled.
Testing	At least 95% approved test cases pass; all critical scenario tests pass.
Usability	SUS completed and findings addressed or documented.
Security	Role/ownership tests pass; secrets and production configuration reviewed.
Data	Course list, rules, questionnaire and test cases approved.
Operations	Backup/restore, deployment and rollback tested.
Documentation	User/admin manuals, API docs, ERD, test report and algorithm explanation completed.

13. 18-Week Agile Roadmap

The proposal uses Agile phases of planning, designing, development, testing, deployment, review and launch. The roadmap below converts those phases into nine two-week sprints. The dates may be shifted to match the academic calendar, but the dependency order should remain.

Sprint	Time	Goal	Major work	Deliverable
Sprint 0	Weeks 1-2	Discovery and requirements baseline	Client interview follow-up; confirm roles, admission score format, 2.50 rule, course list, RIASEC items/scoring, reports; resolve React/database/ML inconsistencies.	Signed requirements, scope, data samples, decision register, risk log.
Sprint 1	Weeks 3-4	UX, diagrams and project foundation	Use case, DFD, ERD, architecture, wireframes; repositories; CI; local environments; database migrations; authentication skeleton.	Approved designs, React/Laravel base, roles and users.

Sprint	Time	Goal	Major work	Deliverable
Sprint 2	Weeks 5-6	Student profile and admission cycle	Profile forms, application state, admission cycles, admin applicant list, validation, fixtures and tests.	Profile/application module completed.
Sprint 3	Weeks 7-8	Exam result processing	Manual entry, CSV preview/import, verification, correction history, permissions and audit events.	Verified exam workflow and import report.
Sprint 4	Weeks 9-10	RIASEC assessment	Questionnaire versioning, 42 items, autosave, submit, scoring, tie handling, result page and unit tests.	Validated RIASEC module.
Sprint 5	Weeks 11-12	Course rules and baseline recommendations	Course catalog, weights, cutoffs, eligibility service, interest fit, ranking, explanations, immutable run storage.	Top recommendations generated from approved cases.
Sprint 6	Weeks 13-14	Admin review, reports and optional ML experiment	Applicant detail, report PDFs/CSV, decision capture, dashboards; train/compare Decision Tree and Random Forest only if dataset is approved.	Operational reports; model comparison/model card or documented deferral.
Sprint 7	Weeks 15-16	Hardening, integration and UAT	E2E/scenario tests, accessibility, performance, security, bug fixing, staging deployment, SUS and client review.	UAT report, SUS result, release candidate.
Sprint 8	Weeks 17-18	Production launch and defense package	Production deployment, backup/restore, manuals, training, source code tag, presentation/demo data, final review and launch.	Production release, turnover package, defense-ready evidence.

13.1 Sprint dependency map

- Requirements and official data must be approved before the recommendation algorithm is considered valid.
- Questionnaire and course mappings must be versioned before test cases are finalized.
- Admission result verification must exist before realistic recommendation end-to-end testing.
- Reports depend on stable applicant, score and recommendation schemas.
- ML is not on the critical path; if data is weak, defer it rather than delay a working explainable MVP.
- Production deployment is blocked until staging UAT and backup restore pass.

13.2 Sprint review checklist

Review area	Required evidence
Demo	Show working software, not screenshots only.
Acceptance	Review user-story criteria and test evidence.
Data validation	Confirm sample data and outputs with client/psychometrician.
Technical quality	Tests pass, code reviewed, migrations reversible, no secrets committed.
Documentation	Update diagrams, API docs, database dictionary and changelog.
Next sprint	Refine backlog, dependencies, risks and owner for each item.

13.3 Milestone deliverables

Milestone	Required outcome
M1 End Week 2	Approved requirements and technology decisions.
M2 End Week 4	Approved UI/ERD/architecture and working authentication skeleton.
M3 End Week 8	Complete profile and official exam-processing workflow.
M4 End Week 10	Validated RIASEC scoring and student result page.
M5 End Week 12	Explainable recommendation engine accepted against validation cases.

Milestone	Required outcome
M6 End Week 14	Reports/dashboard complete; ML experiment decision documented.
M7 End Week 16	UAT/SUS and release candidate.
M8 End Week 18	Production launch, manuals, source code and defense evidence.

14. Team Workflow and Project Management

14.1 Suggested five-member responsibility split

Role slot	Primary ownership
Project Manager / Business Analyst	Client coordination, scope, backlog, diagrams, acceptance criteria, meeting notes and final documentation.
React Frontend Lead	Design system, routes, student screens, accessibility, frontend tests and integration.
Laravel Backend/API Lead	Authentication, controllers, policies, services, API documentation and report jobs.
Database / DevOps Lead	ERD, migrations, seeders, imports, hosting, backups, CI/CD and monitoring.
Recommendation / QA Lead	RIASEC scoring, course fit logic, dataset/model experiment, test plan, validation evidence and defect tracking.

Map the five named proponents to these roles based on strengths. Every critical module should still have a reviewer other than its author.

14.2 Git branching and review

```
main                # protected production-ready branch
develop             # optional integration branch
feature/FR-05-assessment-autosave
feature/FR-09-recommendation-engine
fix/SBT-04-threshold-boundary
chore/database-seeders

Pull request requirements:
- linked user story / issue
- summary and screenshots for UI changes
- migration notes
- tests added or updated
- reviewer approval
- CI passes
- no secrets or generated dependencies committed
```

14.3 Meeting rhythm

Meeting	Duration	Output
Daily/3x weekly stand-up	10-15 minutes	Yesterday, today, blocker; update task board.
Weekly client check-in	30-45 minutes	Clarifications, data/rule approvals, demo feedback.
Sprint planning	60 minutes	Select stories, estimate, define owners and dependencies.
Sprint review	45-60 minutes	Demonstrate accepted functionality and collect feedback.
Sprint retrospective	30 minutes	What worked, what failed, one improvement action.
Technical review	As needed	Architecture, migration, security or algorithm decision.

14.4 Documentation that must stay synchronized

- Software Requirements Specification / approved backlog.
- Use Case Diagram, DFD, ERD, architecture and system flowchart.
- Database migrations and data dictionary.
- OpenAPI endpoint documentation.
- RIASEC scoring specification, course mappings and algorithm version.
- Test cases, defect log, UAT and SUS results.
- Deployment guide, backup/restore guide and user manuals.
- Capstone manuscript chapters and screenshots.

15. Deployment, Operations, and Maintenance

15.1 Deployment topology

Component	Deployment requirement
React web	Build static assets; host on the main domain or subdomain; configure API base URL and cache headers.
Laravel API	PHP runtime, web server, environment variables, queue worker, scheduled tasks, storage permissions and HTTPS.
Database	Managed PostgreSQL/MySQL; restricted network/user; migrations; automated backups.
Reports/files	Private application storage; signed/authorized download endpoints; retention policy.
Domain	Example separation: app.school-domain / api.school-domain, or same domain with /api.
Monitoring	Application errors, failed jobs, disk/database usage, uptime and security events.

15.2 Deployment checklist

1. Confirm hosting supports the selected database, PHP runtime, queue/scheduler and required extensions.
2. Create separate production credentials; disable debug mode; set correct application URL and trusted origins.
3. Run database migrations and approved seed data; never use sample passwords in production.
4. Build React production assets and verify API endpoint configuration.
5. Configure HTTPS, secure cookies, CORS/CSRF, rate limiting and file permissions.
6. Create the first admin account through a secure one-time process.
7. Run smoke tests: login, profile, exam, assessment, recommendation and report.
8. Run backup immediately after launch and verify restore instructions.
9. Tag the source-code release and save migration/version information.
10. Provide user training, support contact and incident escalation steps.

15.3 Backup and recovery plan

Asset	Schedule	Recovery test
Database backup	Daily automated; more frequent during admission peak if supported.	Restore to staging monthly or before launch; verify counts and relationships.
Uploaded/generated files	Daily or synchronized storage backup.	Verify report access and checksums after restore.
Source code	Remote Git repository with protected branches and release tags.	Clone and build from a clean machine.
Configuration	Document non-secret configuration; store secrets securely.	Recreate staging from documentation.

Asset	Schedule	Recovery test
Rollback	Keep prior release artifact and reversible migration plan.	Practice rollback in staging for risky releases.

15.4 Maintenance categories

Type	Examples
Corrective	Fix defects, failed reports, broken imports and security issues.
Adaptive	Update admission cycles, course offerings, thresholds and hosting/runtime changes.
Perfective	Improve usability, performance, reports and explanation quality.
Preventive	Dependency updates, database optimization, backup tests and log review.

16. Risk Register and Mitigation Plan

ID	Risk	Likelihood	Impact	Mitigation
R-01	Unconfirmed admission threshold or grading direction	High	High	Obtain written rule and boundary examples before Sprint 5; implement configurable rules.
R-02	No valid ML training labels	High	High	Use explainable baseline; collect expert-reviewed cases; document ML as experimental or defer.
R-03	MySQL/PostgreSQL inconsistency	High	Medium	Make D-03 decision in Sprint 0 and update proposal, ERD, hosting and code.
R-04	RIASEC question/scoring changes without approval	Medium	High	Version questionnaire; approval workflow; immutable completed sessions.
R-05	Students enter inaccurate official scores	High	High	Staff-only verification/import; show verification status; audit changes.
R-06	Course mapping is subjective or incomplete	High	High	Workshop with psychometrician; document weights/rationale; validate sample cases.
R-07	Scope expands to mobile app or enrollment system	Medium	High	Protect MVP scope; responsive web only; integrations moved to Phase 2.
R-08	Data exposure due to broken access control	Medium	High	Backend policies, ownership tests, audit logs, staging security testing.
R-09	Host limitations or deployment failure	Medium	High	Confirm plan early; staging deployment by Sprint 6; prepare alternative host.
R-10	Team merge conflicts and uneven workload	High	Medium	Feature ownership, small pull requests, code reviews, task board and weekly integration.
R-11	Insufficient time for UAT/SUS	Medium	High	Freeze features after Sprint 6; reserve Sprints 7-8 for validation and release.
R-12	Capstone document and system diverge	High	Medium	Update manuscript, diagrams, schema and screenshots at each milestone.

16.1 Top three risks to resolve first

1. Institutional policy
Confirm the official examination score format, “2.50 or better” comparison direction, course-specific cutoffs, board-course rules and who may verify scores.

2. Recommendation validity
Obtain the final RIASEC question mapping and psychometrician-approved course profiles. Without these, the system can function technically but the recommendations cannot be defended.

3. Machine-learning evidence
Define the label and dataset before promising ML accuracy. A transparent baseline is a valid system; an unvalidated model is a project risk.

17. Deliverables and Defense Readiness

17.1 Required technical deliverables

Deliverable	Evidence
Source code	React frontend, Laravel API, optional ML workspace, environment examples and tagged release.
Database	ERD, migrations, seeders, data dictionary and backup/restore procedure.
Diagrams	Use Case, DFD, ERD, system flowchart, sequence diagrams and deployment architecture.
Requirements	Approved scope, roles, business rules, user stories and acceptance criteria.
Algorithm evidence	RIASEC scoring specification, course weights, eligibility rules, validation case set, comparison results and model card if ML is used.
Testing	Unit/API/E2E results, scenario-based test cases, defect log, performance/security findings, UAT and SUS result.
Deployment	Production/staging URLs, deployment checklist, release notes, backups and monitoring.
Documentation	Student manual, administrator manual, API documentation, installation/maintenance guide and turnover checklist.

17.2 Recommended defense demonstration script

1. Show the problem and system scope in one minute.
2. Login as a student and show the dashboard completion checklist.
3. Login as staff and verify/import the applicant's official exam result.
4. Complete or use a prepared RIASEC assessment and show six category scores/top code.
5. Generate recommendations and explain the exact interest and eligibility factors.
6. Open a course detail and show why another course is ineligible or ranked lower.
7. Record the student decision and generate the guidance report.
8. Show admin rules/weights, versioning and audit log.
9. Present test evidence, SUS result, performance and algorithm validation.
10. Explain limitations: guidance only, TCC courses only, input honesty and no long-term outcome tracking.

17.3 Questions the panel may ask

Likely question	Defensible answer
Why React?	Reusable responsive interface, clear separation from backend and better management of complex forms/state.

Likely question	Defensible answer
Why not React only?	React is a client interface; authentication, official data, database transactions, recommendation logic and reports require a backend.
Why RIASEC?	It directly represents vocational interests and is the framework selected by the proposal.
How is the recommendation calculated?	Show the six scores, course weights, eligibility rule, composite formula and stored explanation.
Is it really machine learning?	Answer honestly: baseline is deterministic; ML is included only if trained on documented approved labels and evaluated against the baseline.
How do you know the recommendation is correct?	Manual scoring tests, psychometrician-approved cases, top-k agreement and explanation review.
How do you protect student data?	Backend authorization, data minimization, HTTPS, secure authentication, audit logs and backups.
What happens when policies change?	Admission cycles, versioned rules, questionnaires and immutable historical recommendation snapshots.
Can students ignore the result?	Yes. It is a decision-support recommendation and not a final enrollment decision.

18. Proposal Corrections and Items to Confirm

18.1 Technical consistency corrections

Item	Required correction
Title spelling	“COLLOGE” should be corrected to “COLLEGE”.
Methodology heading	“RESEACH METHODOLOGY” should be “RESEARCH METHODOLOGY”.
Frontend stack	Replace or expand Bootstrap-only references to describe React as the presentation layer.
Database	Choose MySQL or PostgreSQL and use it consistently in methodology, architecture and deployment.
Architecture scope	Remove “mobile app” unless PWA/native mobile is explicitly included.
Machine learning	Add dataset source, target label, features, preprocessing, split, metrics, model comparison and governance.
User roles	Separate Guidance/Psychometrician, Admission/Testing and System Admin permissions.
Exam data	Clarify who encodes and verifies official scores; students should not self-verify.
Course rules	Document course-specific thresholds and confirm the exact meaning of 2.50 or better.
RIASEC scoring	Add question-to-category mapping, response scale, maximum score, tie handling and questionnaire version.
Tense consistency	Proposal/future tense and completed-system/past tense are mixed; revise according to current project stage.
Duplicate objective paragraph	The general objective appears twice and should be consolidated.
References	Verify every citation, publication year, URL, author and access before final submission.

18.2 Client confirmation checklist before coding

- ☐ Official list of TCC courses and codes for the target admission cycle.
- ☐ Which courses are board-regulated and their exact admission thresholds.
- ☐ Admission exam field format, scale, comparison direction and correction policy.
- ☐ Official 42 RIASEC statements, category mapping and scoring/interpretation rules.

- ☐ Whether a student can retake the assessment and who approves a reset.
- ☐ Course-to-RIASEC profiles/weights and who signs them off.
- ☐ Required student profile fields and which are optional.
- ☐ Required report layout, signatories and export formats.
- ☐ User roles, named office owners and approval permissions.
- ☐ Hosting/database capabilities, domain and production support owner.
- ☐ Availability and legal/ethical usability of historical data for ML.
- ☐ Target dates for UAT, SUS, launch and capstone defense.

Appendix A. Prioritized Product Backlog

Priority	ID	User story	Acceptance summary
P0	US-01	As a student, I can register and log in securely so I can access my application.	Valid data creates account; duplicate email rejected; protected pages require login.
P0	US-02	As a student, I can complete and update my profile.	Required fields validated; completeness shown; official identifiers protected.
P0	US-03	As Admission staff, I can encode and verify exam results.	Only authorized role; value/range validated; verification and audit recorded.
P0	US-04	As staff, I can import exam results through CSV preview.	Invalid/duplicate rows identified; confirmation required; batch report generated.
P0	US-05	As a student, I can start/resume the active RIASEC assessment.	Correct version loaded; one response per item; autosave and progress work.
P0	US-06	As a student, I can submit only when required items are complete.	Missing items shown; completed attempt becomes read-only.
P0	US-07	As Guidance staff, I can validate questionnaire versions.	Draft/approved/active states; only one active version per policy; changes audited.
P0	US-08	As the system, I calculate six RIASEC scores and top code.	Matches manual gold cases; tie policy deterministic; scoring version stored.
P0	US-09	As Admin, I can manage courses, rules and weights.	Validation prevents missing dimensions/invalid thresholds; versions effective by cycle.
P0	US-10	As a student, I receive ranked course recommendations.	Only after prerequisites; top items include scores, eligibility and explanation.
P0	US-11	As Guidance staff, I can review an applicant's full recommendation record.	Profile, exam, scores, run version and decision visible with proper authorization.
P0	US-12	As a student, I can record accept/reject/undecided.	Decision is separate from ranking and timestamped.
P0	US-13	As Guidance staff, I can generate a printable recommendation report.	Authorized PDF contains applicant, scores, top courses, reasons and disclaimer.
P0	US-14	As Admin, I can view audit logs.	Filters by actor/action/date/entity; logs are not editable through UI.
P1	US-15	As staff, I can view aggregate dashboards and export CSV.	Counts reconcile with filtered records; large results are paginated/queued.
P1	US-16	As Admin, I can manage admission cycles and activate current configuration.	Only one relevant active cycle; old records remain linked to old cycle.
P1	US-17	As a user, I receive clear loading/error/empty states.	No blank screens; retry available; errors are logged with request ID.
P1	US-18	As a keyboard user, I can complete the assessment and navigate core pages.	Logical tab order, focus visible, labels and text alternatives.
P2	US-19	As Guidance staff, I can compare recommendation versions.	Old/new algorithm inputs and rank changes shown.
P2	US-20	As an authorized analyst, I can evaluate an ML model version.	Dataset provenance, metrics, approval and rollback documented.

Appendix B. Starter PostgreSQL Schema

This is a starter schema for planning and migration design. The Laravel migrations should be the authoritative implementation. Names and fields must be adjusted after the client confirms official data requirements.

B.1 Identity, application, exam and course tables

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE roles (
  id SMALLSERIAL PRIMARY KEY,
  code VARCHAR(30) NOT NULL UNIQUE,
  name VARCHAR(80) NOT NULL
);

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  role_id SMALLINT NOT NULL REFERENCES roles(id),
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (status IN ('pending','active','locked','inactive'))
);

CREATE TABLE student_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL UNIQUE REFERENCES users(id),
  applicant_no VARCHAR(40) UNIQUE,
  first_name VARCHAR(100) NOT NULL,
  middle_name VARCHAR(100),
  last_name VARCHAR(100) NOT NULL,
  birth_date DATE,
  contact_no VARCHAR(30),
  address_json JSONB,
  shs_strand VARCHAR(60),
  school_name VARCHAR(180),
  consent_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE admission_cycles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(80) NOT NULL,
  academic_year VARCHAR(20) NOT NULL,
  starts_at DATE NOT NULL,
  ends_at DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'draft',
  CHECK (status IN ('draft','open','closed','archived')),
  CHECK (ends_at >= starts_at)
);

CREATE TABLE courses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code VARCHAR(30) NOT NULL UNIQUE,
  name VARCHAR(180) NOT NULL,
  department VARCHAR(120),
  description TEXT,
  board_regulated BOOLEAN NOT NULL DEFAULT FALSE,
  requirements_text TEXT,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE applications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_profile_id UUID NOT NULL REFERENCES student_profiles(id),
  admission_cycle_id UUID NOT NULL REFERENCES admission_cycles(id),
  application_no VARCHAR(50) NOT NULL UNIQUE,
  preferred_course_id UUID REFERENCES courses(id),
  status VARCHAR(40) NOT NULL DEFAULT 'DRAFT',
  submitted_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (student_profile_id, admission_cycle_id)
```

```
);

CREATE TABLE admission_exam_results (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID NOT NULL UNIQUE REFERENCES applications(id),
  raw_score NUMERIC(10,2),
  grade_equivalent NUMERIC(5,2) NOT NULL,
  source VARCHAR(30) NOT NULL,
  verification_status VARCHAR(20) NOT NULL DEFAULT 'pending',
  verified_by UUID REFERENCES users(id),
  verified_at TIMESTAMPTZ,
  remarks TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (source IN ('manual','csv_import','integration')),
  CHECK (verification_status IN ('pending','verified','rejected'))
);
```

B.2 RIASEC assessment and course-fit tables

```
CREATE TABLE riasec_questionnaires (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(120) NOT NULL,
  version VARCHAR(30) NOT NULL UNIQUE,
  instructions TEXT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'draft',
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (status IN ('draft','approved','active','retired'))
);

CREATE TABLE riasec_questions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  questionnaire_id UUID NOT NULL REFERENCES riasec_questionnaires(id),
  sequence_no SMALLINT NOT NULL,
  statement TEXT NOT NULL,
  dimension CHAR(1) NOT NULL,
  reverse_scored BOOLEAN NOT NULL DEFAULT FALSE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (questionnaire_id, sequence_no),
  CHECK (dimension IN ('R','I','A','S','E','C'))
);

CREATE TABLE assessment_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID NOT NULL REFERENCES applications(id),
  questionnaire_id UUID NOT NULL REFERENCES riasec_questionnaires(id),
  status VARCHAR(20) NOT NULL DEFAULT 'in_progress',
  attempt_no SMALLINT NOT NULL DEFAULT 1,
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ,
  submitted_ip INET,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (status IN ('in_progress','completed','void')),
  UNIQUE (application_id, questionnaire_id, attempt_no)
);

CREATE TABLE assessment_responses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES assessment_sessions(id) ON DELETE CASCADE,
  question_id UUID NOT NULL REFERENCES riasec_questions(id),
  response_value SMALLINT NOT NULL,
  answered_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (session_id, question_id),
  CHECK (response_value IN (0,1))
);

CREATE TABLE riasec_scores (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL UNIQUE REFERENCES assessment_sessions(id),
  r_score SMALLINT NOT NULL,
  i_score SMALLINT NOT NULL,
  a_score SMALLINT NOT NULL,
  s_score SMALLINT NOT NULL,
  e_score SMALLINT NOT NULL,
  c_score SMALLINT NOT NULL,
  top_code VARCHAR(12) NOT NULL,
  normalized_scores JSONB NOT NULL,
  scoring_version VARCHAR(30) NOT NULL,
  calculated_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);

CREATE TABLE course_riasec_weights (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID NOT NULL REFERENCES courses(id),
  dimension CHAR(1) NOT NULL,
  weight NUMERIC(6,5) NOT NULL,
  version VARCHAR(30) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (course_id, dimension, version),
  CHECK (dimension IN ('R','I','A','S','E','C')),
  CHECK (weight >= 0 AND weight <= 1)
);

CREATE TABLE course_admission_rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID NOT NULL REFERENCES courses(id),
  admission_cycle_id UUID NOT NULL REFERENCES admission_cycles(id),
  score_field VARCHAR(30) NOT NULL DEFAULT 'grade_equivalent',
  operator VARCHAR(10) NOT NULL,
  threshold_value NUMERIC(8,2) NOT NULL,
  is_mandatory BOOLEAN NOT NULL DEFAULT TRUE,
  notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (operator IN ('<=','>=','<','>','='))
);
```

B.3 Model, recommendation, decision and audit tables

```
CREATE TABLE model_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(120) NOT NULL,
  version VARCHAR(40) NOT NULL,
  algorithm VARCHAR(40) NOT NULL,
  training_dataset_ref TEXT NOT NULL,
  feature_schema JSONB NOT NULL,
  metrics JSONB NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'draft',
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (name, version),
  CHECK (status IN ('draft','validated','active','retired'))
);

CREATE TABLE recommendation_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID NOT NULL REFERENCES applications(id),
  algorithm_type VARCHAR(30) NOT NULL,
  algorithm_version VARCHAR(40) NOT NULL,
  model_version_id UUID REFERENCES model_versions(id),
  input_snapshot JSONB NOT NULL,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (algorithm_type IN ('rules','decision_tree','random_forest','hybrid'))
);

CREATE TABLE recommendation_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id UUID NOT NULL REFERENCES recommendation_runs(id) ON DELETE CASCADE,
  course_id UUID NOT NULL REFERENCES courses(id),
  rank SMALLINT NOT NULL,
  interest_score NUMERIC(6,3) NOT NULL,
  academic_score NUMERIC(6,3) NOT NULL,
  final_score NUMERIC(6,3) NOT NULL,
  eligibility_status VARCHAR(20) NOT NULL,
  explanation JSONB NOT NULL,
  UNIQUE (run_id, rank),
  UNIQUE (run_id, course_id),
  CHECK (eligibility_status IN ('eligible','conditional','ineligible'))
);

CREATE TABLE student_decisions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id UUID NOT NULL REFERENCES recommendation_runs(id),
  selected_course_id UUID REFERENCES courses(id),
  action VARCHAR(20) NOT NULL,
  reason TEXT,
  decided_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (action IN ('accepted','rejected','undecided','chose_other'))
);
```

```
CREATE TABLE audit_logs (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  action VARCHAR(100) NOT NULL,
  entity_type VARCHAR(80),
  entity_id UUID,
  old_values JSONB,
  new_values JSONB,
  ip_address INET,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_applications_cycle_status
  ON applications(admission_cycle_id, status);
CREATE INDEX idx_sessions_application_status
  ON assessment_sessions(application_id, status);
CREATE INDEX idx_recommendation_items_run_rank
  ON recommendation_items(run_id, rank);
CREATE INDEX idx_audit_entity_date
  ON audit_logs(entity_type, entity_id, created_at DESC);
```

Appendix C. Test Case and Acceptance Templates

C.1 Scenario-based test case template

Field	Entry
Test Case ID	SBT-___
Module / Requirement	FR-___ / US-___
Title	
Preconditions	
Test data	
Steps	1. 2. 3.
Expected result	
Actual result	
Status	Pass / Fail / Blocked
Evidence	Screenshot, API response, log or report reference
Tester / Date	
Defect ID	If failed

C.2 User acceptance record

Field	Required record
UAT session	Date, location, build/version
Participant role	Student / Guidance / Admission / Admin
Tasks	List tasks attempted
Completion	Completed / completed with assistance / failed
Observed issues	Navigation, terminology, speed, errors
SUS response	Attach anonymous SUS form/result
Required changes	Prioritized findings
Client decision	Accepted / conditional acceptance / rejected
Sign-off	Authorized representative and date

C.3 Algorithm validation case template

Field	Value
Case ID	
Questionnaire version	

Field	Value
R/I/A/S/E/C raw scores	
Normalized vector	
Verified admission result	
Expected eligible courses	
Expected top 3 / acceptable set	
Expected rationale	
Actual result and scores	
Agreement / discrepancy	
Psychometrician reviewer	
Approval date	
Algorithm version	

Appendix D. Definition of Done and Checklists

D.1 Definition of Done for a user story

- ☐ Acceptance criteria reviewed and satisfied.
- ☐ Frontend and backend validation implemented where applicable.
- ☐ Authorization/ownership checks tested.
- ☐ Database migration/seed changes reviewed and reversible.
- ☐ Unit/feature/component tests added and passing.
- ☐ Loading, empty, validation and error states implemented.
- ☐ Responsive and keyboard-accessible behavior checked.
- ☐ No secrets, debug statements or personal test data committed.
- ☐ API/ERD/user documentation updated.
- ☐ Pull request approved and merged; staging smoke test completed.

D.2 Release readiness checklist

- ☐ Approved database backup exists and restore has been tested.
- ☐ Production environment variables and HTTPS are configured.
- ☐ Debug mode disabled; logs and failed-job monitoring enabled.
- ☐ Migrations and seed data rehearsed in staging.
- ☐ Core end-to-end smoke tests pass in production-like environment.
- ☐ Admin and staff accounts verified; default/sample accounts removed.
- ☐ User/admin manuals and support contact are available.
- ☐ Release tag, changelog and rollback plan saved.
- ☐ Client launch approval recorded.

D.3 Immediate next actions for the team

Order	Action
1	Schedule a requirements validation meeting with Guidance/Psychometrician and Admission/Testing.
2	Choose PostgreSQL or MySQL and update the proposal consistently.
3	Obtain official course list, admission rules, RIASEC question mapping and sample exam data.
4	Create the repository, issue board, coding conventions and protected branches.
5	Finalize use case, DFD, ERD, architecture and wireframes before feature coding.
6	Build the rules-based recommendation baseline and validation cases before ML.
7	Deploy a staging skeleton by Sprint 2 so hosting problems are found early.
8	Reserve the final four weeks for testing, UAT, SUS, deployment and defense evidence.

Final roadmap recommendation — Build the secure, explainable system first: React UI, Laravel API, a relational system of record, and machine learning only as a validated enhancement.